HOA SEN
UNIVERSITY

Lecture 4
SQL 3 – Select, Grouping data

ĐẠI HỌC HOA SEN

Objectives

More Complex SQL Queries:

- Nesting Of Queries
- Joined Relation
- Exists Function
- Aggregate Functions
- Grouping – Having Clause
- Substring Comparison – Arithmetic Operations
- Order By Clause

- Reference: Chapter 7

ĐẠI HỌC HOA SEN

Set Operations

- SQL has directly incorporated some set operations
- There is a union operation (UNION), and in *some versions* of SQL there are set difference (MINUS) and intersection (INTERSECT) operations
- The resulting relations of these set operations are sets of tuples; *duplicate tuples are eliminated from the result*
- The set operations apply only to *union compatible relations*; the two relations must have the same attributes and the attributes must appear in the same order

Set Operations - Example

- Q1 : Make a list of all project numbers for projects that involve an employee whose last name is 'Smith' as a worker or as a manager of the department that controls the project.

```
(SELECT     Pname
  FROM      PROJECT, DEPARTMENT, EMPLOYEE
  WHERE          Dnum=Dnumber AND Mgr_ssn=Ssn AND
            Lname='Smith')
  UNION
(SELECT     Pname
  FROM      PROJECT, WORKS_ON, EMPLOYEE
  WHERE          Pnumber=Pno AND Essn=Ssn AND
            Lname='Smith')
```

ALL – Union, Except, Intersect

(a)

R

A
a1
a2
a2
a3

S

A
a1
a2
a4
a5

(b)

T

A
a1
a1
a2
a2
a2
a3
a4
a5

(c)

T

A
a2
a3

(d)

T

A
a2
a3

Figure 8.5
The results of SQL multiset operations.
(a) Two tables, R(A) and S(A).
(b) R(A) UNION ALL S(A).
(c) R(A) EXCEPT ALL S(A).
(d) R(A) INTERSECT ALL S(A).

Nesting Of Queries - Subqueries

- A complete SELECT query, called a *nested query*, can be specified within the WHERE-clause of another query, called the *outer query*
  - Many of the previous queries can be specified in an alternative form using nesting

- Q2: Retrieve the name and address of all employees who work for the 'Research' department.

```
SELECT Fname, Lname, Address
  FROM    EMPLOYEE
  WHERE   Dno IN
          (SELECT Dnumber
          FROM    DEPARTMENT
          WHERE   Dname='Research' )
```

Nesting Of Queries (2)

- The nested query selects the number of the 'Research' department
- The outer query select an EMPLOYEE tuple if its DNO value is in the result of either nested query
- The comparison operator IN compares a value v with a set (or multi-set) of values V, and evaluates to TRUE if v is one of the elements in V
- In general, we can have several levels of nested queries
- A reference to an *unqualified attribute* refers to the relation declared in the *innermost nested query*
- In this example, the nested query is *not correlated* with the outer query

Correlated Nested Queries

- If a condition in the WHERE-clause of a *nested query* references an attribute of a relation declared in the *outer query*, the two queries are said to be *correlated*
  - The result of a correlated nested query is different for each tuple (or combination of tuples) of the relation(s) the outer query

- Q3: Retrieve the name of each employee who has a dependent with the same first name as the employee.

```
SELECT      E.Fname, E.Lname
    FROM    EMPLOYEE AS E
    WHERE   E.Ssn IN
            (SELECT       Essn
                FROM      DEPENDENT
                WHERE     Essn = E.Ssn AND
                 E.Fname = Dependent_name)
```

Correlated Nested Queries (2)

- In Q3, the nested query has a different result in the outer query
- A query written with nested SELECT... FROM... WHERE... blocks and using the = or IN comparison operators can ***always*** be expressed as a single block query. For example, Q3 may be written as in Q3A

Q3A:
```
SELECT   E.Fname, E.Lname
FROM     EMPLOYEE E, DEPENDENT D
WHERE    E.Ssn=D.Essn AND
E.Fname=D.Dependent_name
```

Correlated Nested Queries (3)

- The original SQL as specified for SYSTEM R also had a **CONTAINS** comparison operator, which is used in conjunction with nested correlated queries
  - This operator was *dropped from the language*, possibly because of the difficulty in implementing it efficiently
  - Most implementations of SQL do not have this operator
  - The CONTAINS operator compares *two sets of values*, and returns TRUE if one set contains all values in the other set
    - Reminiscent of the division operation of algebra

Correlated Nested Queries (4)

- Q4: Retrieve the name of each employee who works on all the projects controlled by department number 5.

```
SELECT     Fname, Lname
  FROM     EMPLOYEE
  WHERE   ((SELECT  Pno
            FROM     WORKS_ON
            WHERE    Ssn=Essn)
            CONTAINS
                 (SELECT  Pnumber
                 FROM     PROJECT
                 WHERE         Dnum=5))
```

Correlated Nested Queries (5)

- In Q4, the second nested query, which is *not correlated* with the outer query, retrieves the project numbers of all projects controlled by department 5

- The first nested query, which is correlated, retrieves the project numbers on which the employee works, which is *different for each employee tuple* because of the correlation

The EXISTS Function

- EXISTS is used to check whether the result of a correlated nested query is empty (contains no tuples) or not
  - We can formulate Query 3 in an alternative form that uses EXISTS as Q3B

The EXISTS Function (2)

- Query 3: Retrieve the name of each employee who has a dependent with the same first name as the employee.

Q3B:
```
SELECT   Fname, Lname
FROM     EMPLOYEE
WHERE    EXISTS
         (SELECT *
         FROM     DEPENDENT
         WHERE    Ssn=Essn AND
                  Fname=Dependent_name)
```

The EXISTS Function (3)

- Query 5: Retrieve the names of employees who have no dependents.

Q5:
```
SELECT   Fname, Lname
FROM     EMPLOYEE
WHERE    NOT EXISTS
         (SELECT *
         FROM     DEPENDENT
         WHERE    Ssn=Essn)
```

- In Q5, the correlated nested query retrieves all DEPENDENT tuples related to an EMPLOYEE tuple. If *none exist*, the EMPLOYEE tuple is selected
  - EXISTS is necessary for the expressive power of SQL

ĐẠI HỌC HOA SEN

Joined Relations Feature in SQL2

- Can specify a "joined relation" in the FROM-clause
  - Looks like any other relation but is the result of a join
  - Allows the user to specify different types of joins (regular "theta" JOIN, NATURAL JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, CROSS JOIN, etc)

Joined Relations Feature in SQL2 (2)

- Examples:

  Q7:

```
SELECT    E.Fname, E.Lname, S.Fname, S.Lname
 FROM          EMPLOYEE E S
 WHERE         E.super_ssn=S.ssn
```

- can be written as:

  Q7b:

```
SELECT    E.Fname, E.Lname, S.Fname, S.Lname
 FROM     EMPLOYEE E LEFT OUTER JOIN   EMPLOYEE
S ON  (E.Super_ssn=S.Ssn)
```

Joined Relations Feature in SQL2 (3)

- Examples:
  Q8:

```
SELECT    Fname, Lname, Address
 FROM     EMPLOYEE, DEPARTMENT
 WHERE    Dname='Research' AND Dnumber=Dno
```

- could be written as:
  Q8A:

```
SELECT Fname, Lname, Address
   FROM (EMPLOYEE JOIN DEPARTMENT ON Dnumber=Dno)
   WHERE Dname='Research'
```

- or as:
  Q8B:

```
SELECT Fname, Lname, Address
   FROM (EMPLOYEE NATURAL JOIN EPARTMENT
         AS DEPT(Dname, Dno, Mssn,Msdate)
   WHERE Dname='Research'
```

Joined Relations Feature in SQL2 (4)

- Another Example:

  Q9: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birthdate.

```
SELECT      Pnumber, Dnum, Lname, Bdate, Address
   FROM     PROJECT, DEPARTMENT, EMPLOYEE
   WHERE    Dnum=Dnumber AND Mgr_ssn=Ssn
            AND Plocation='Stafford'
```

Q2 could be written as follows; this illustrates multiple joins in the joined tables

```
SELECT Pnumber, Dnum, Lname, Bdate, Address
   FROM (PROJECT JOIN DEPARTMENT ON
   (Dnum=Dnumber) JOIN EMPLOYEE ON
   (Mgr_ssn=Ssn) )
   WHERE Plocation='Stafford'
```

Aggregate Functions

- Include **COUNT, SUM, MAX, MIN, and AVG**
- Query 10: Find the maximum salary, the minimum salary, and the average salary among all employees.

```
SELECT MAX(Salary), MIN(Salary), AVG(Salary)
FROM   EMPLOYEE
```

- Some SQL implementations *may not allow more than one function* in the SELECT-clause

ĐẠI HỌC HOA SEN

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

```
SELECT MAX(Salary), MIN(Salary), AVG(Salary)
 FROM  EMPLOYEE
```

Result?

Aggregate Functions (2)

- Query 11: Find the maximum salary, the minimum salary, and the average salary among employees who work for the 'Research' department.

```
Q16:
SELECT  MAX(Salary), MIN(Salary), AVG(Salary)
FROM    EMPLOYEE, DEPARTMENT
WHERE   Dno=Dnumber AND Dname='Research'
```

EMPLOYEE

Salary	Super_ssn	Dno
30000	333445555	5
40000	888665555	5
25000	987654321	4
43000	888665555	4
38000	333445555	5
25000	333445555	5
25000	987654321	4
55000	NULL	1

DEPARTMENT

Dname	Dnumber
Research	5
Administration	4
Headquarters	1

Result?

Aggregate Functions (3)

- Queries 12: Retrieve the total number of employees in the company.

```
Q12:   SELECT   COUNT (*)
       FROM     EMPLOYEE
```

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Result?

Aggregate Functions (3)

- Queries 13: the number of employees in the 'Research' department.

```
Q13:   SELECT   COUNT (*)
       FROM     EMPLOYEE, DEPARTMENT
       WHERE    Dno=Dnumber AND Dname='Research'
```

EMPLOYEE

Fname	Minit	Lname	Dno
John	B	Smith	5
Franklin	T	Wong	5
Alicia	J	Zelaya	4
Jennifer	S	Wallace	4
Ramesh	K	Narayan	5
Joyce	A	English	5
Ahmad	V	Jabbar	4
James	E	Borg	1

DEPARTMENT

Dname	Dnumber
Research	5
Administration	4
Headquarters	1

Result?

ĐẠI HỌC HOA SEN

Grouping

- In many cases, we want to apply the aggregate functions to *subgroups of tuples* in a relation
- Each subgroup of tuples consists of the set of tuples that have the *same value* for the *grouping attribute(s)*
- The function is applied to each subgroup independently
- SQL has a **GROUP BY**-clause for specifying the grouping attributes, which *must also appear in the SELECT-clause*

Grouping (2)

- Q14: For each department, retrieve the department number, the number of employees in the department, and their average salary.

```
SELECT Dno, COUNT(*), AVG(Salary)
  FROME MPLOYEE
  GROUP BYDno
```

  - In Q14, the EMPLOYEE tuples are divided into groups-
    - Each group having the same value for the grouping attribute DNO
  - The COUNT and AVG functions are applied to each such group of tuples separately
  - The SELECT-clause includes only the grouping attribute and the functions to be applied on each group of tuples
  - A join condition can be used in conjunction with grouping

Grouping (3)

- Query 15: For each project, retrieve the project number, project name, and the number of employees who work on that project.

```
SELECT     Pnumber, Pname, COUNT (*)
  FROM     PROJECT, WORKS_ON
  WHERE    Pnumber=Pno
  GROUP BY Pnumber, Pname
```

  - In this case, the grouping and functions are applied after the joining of the two relations

ĐẠI HỌC HOA SEN

The HAVING-clause

- Sometimes we want to retrieve the values of these functions for only those *groups that satisfy certain conditions*

- The **HAVING**-clause is used for specifying a selection condition on groups (rather than on individual tuples)

The HAVING-clause (2)

- Query 16: For each project *on which more than two employees work*, retrieve the project number, project name, and the number of employees who work on that project.

```
SELECT Pnumber, Pname, COUNT(*)
    FROM PROJECT, WORKS_ON
    WHERE Pnumber=Pno
    GROUP BY Pnumber, Pname
      HAVING COUNT (*) > 2
```

ĐẠI HỌC HOA SEN

Summary of SQL Queries

- A query in SQL can consist of up to six clauses, but only the first two, SELECT and FROM, are mandatory. The clauses are specified in the following order:

SELECT <attribute list>
FROM <table list>
[**WHERE** <condition>]
[**GROUP BY** <grouping attribute(s)>
[**HAVING** <group condition>]]
[**ORDER BY** <attribute list>]

ĐẠI HỌC HOA SEN

Summary of SQL Queries (2)

- The SELECT-clause lists the attributes or functions to be retrieved
- The FROM-clause specifies all relations (or aliases) needed in the query but not those needed in nested queries

- The WHERE-clause specifies the conditions for selection and join of tuples from the relations specified in the FROM-clause
- GROUP BY specifies grouping attributes
- HAVING specifies a condition for selection of groups

- ORDER BY specifies an order for displaying the result of a query
  - A query is evaluated by first applying the WHERE-clause, then GROUP BY and HAVING, and finally the SELECT-clause

ĐẠI HỌC HOA SEN

Views in SQL

- A view is a “virtual” table that is derived from other tables
- Allows for limited update operations
  - Since the table may not physically be stored
- Allows full query operations
- A convenience for expressing certain operations

Specification of Views

- SQL command: **`CREATE VIEW`**
  - a table (view) name
  - a possible list of attribute names (for example, when arithmetic operations are specified or when we want the names to be different from the attributes in the base relations)
  - a query to specify the table contents

SQL Views: An Example

- Specify a different WORKS_ON table

```
CREATE VIEW WORKS_ON_NEW AS
SELECT Fname, Lname, Pname, Hours
    FROM EMPLOYEE, PROJECT, WORKS_ON
    WHERE Ssn=Essn AND Pno=Pnumber;
```

- Using a Virtual Table
  - We can specify SQL queries on a newly create table (view):

```
SELECT Fname, Lname
     FROM WORKS_ON_NEW
     WHERE Pname='Seena';
```

Drop view

- When no longer needed, a view can be dropped:

```
DROP WORKS_ON_NEW;
```

ĐẠI HỌC HOA SEN

Efficient View Implementation

- Query modification:
  - Present the view query in terms of a query on the underlying base tables
- Disadvantage:
  - Inefficient for views defined via complex queries
    - Especially if additional queries are to be applied to the view within a short time period

Efficient View Implementation

- View materialization:
  - Involves physically creating and keeping a temporary table
- Assumption:
  - Other queries on the view will follow
- Concerns:
  - Maintaining correspondence between the base table and the view when the base table is updated
- Strategy:
  - Incremental update

ĐẠI HỌC HOA SEN

Update Views

- Update on a single view without aggregate operations:
  - Update may map to an update on the underlying base table
- Views involving joins:
  - An update *may* map to an update on the underlying base relations
    - Not always possible

ĐẠI HỌC HOA SEN

Un-updatable Views

- Views defined using groups and aggregate functions are not updateable
- Views defined on multiple tables using joins are generally not updateable
- **`WITH CHECK OPTION`**: must be added to the definition of a view if the view is to be updated
  - To allow check for updatability and to plan for an execution strategy

ĐẠI HỌC
HOA SEN

Q & A